Coding for Artificial Intelligence
Problem Set 3.1
The Sensor Anomaly Filter

Submission Documentation

Name: Tanishka Hira
Problem: anomaly_auditor
Date: 24.08.2026
Language: Python
Module: Coding for Artificial Intelligence

Shutdown Triggered Flag Explanation

The `shutdown_triggered` flag tracks emergency shutdown states. Initialized to `False`, it switches to `True` when the inner loop encounters a `"STOP"` signal, triggering an immediate inner loop `break`. The outer loop evaluates this flag; because it is now `True`, it breaks as well. This explicit `break` bypasses the outer loop's `else` block entirely. Conversely, if no `"STOP"` signal is detected, the flag stays `False`, allowing the outer loop to complete normally and execute its `else` block.

1 Objective

The objective of this practical is to implement a sensor anomaly auditor using nested loops and flow-control statements. The program processes batches of sensor readings, ignores erroneous readings, detects values above the safe threshold, and performs an emergency shutdown when the `"STOP"` signal is encountered.

The program also implements the Rolling Delta check to identify a spike when the difference between two consecutive numeric readings exceeds 5.0.

2 Python Source Code

```
'''
Problem : anomaly_auditor
Author  : Tanishka Hira
Date    : 24.08.2026
'''

telemetry_stream = [
    [22.5, 23.0, 22.8],
    [25.1, "ERR", 24.9],
    [30.2, 35.5, 40.1],
    [22.0, 22.1, "STOP"]
```

```
]

shutdown_triggered = False

# Outer Loop processes each batch
for batch_id in range(len(telemetry_stream)):

    batch = telemetry_stream[batch_id]

    print(f"---- Auditing batch {batch_id}: {batch} ---")

    # Previous numeric reading for rolling delta bonus
    previous_value = None

    # Inner loop processes readings
    for reading in batch:

        # Emergency shutdown
        if reading == "STOP":
            print(f"Emergency shutdown at batch {batch_id}.")
            shutdown_triggered = True
            break

        # Ignore noise
        if reading == "ERR":
            print(f"Noise ignored at batch {batch_id} (ERR).")
            continue

        # Process only numeric reading
        if isinstance(reading, (int, float)):

            # Check anomaly
            if reading > 35.0:
                print(
                    f"Anomaly Detected at batch {batch_id}: "
                    f"{reading}"
                )

            # Rolling delta check
            if previous_value is not None:

                delta = abs(reading - previous_value)

                if delta > 5.0:
                    print(
```

```
                            f"Spike Detected at batch {batch_id}: "
                            f"{previous_value} -> {reading} "
                            f"(Delta {delta:.1f})"
                        )

                # Update previous value only for numeric readings
                previous_value = reading

    # STOP must terminate the outer loop too
    if shutdown_triggered:
        break

else:
    print("Audit Complete: No system-wide failures")
```

3 Program Output

The program produces the following output:

```
---- Auditing batch 0: [22.5, 23.0, 22.8] ---
---- Auditing batch 1: [25.1, 'ERR', 24.9] ---
Noise ignored at batch 1 (ERR).
---- Auditing batch 2: [30.2, 35.5, 40.1] ---
Anomaly Detected at batch 2: 35.5
Spike Detected at batch 2: 30.2 -> 35.5 (Delta 5.3)
Anomaly Detected at batch 2: 40.1
---- Auditing batch 3: [22.0, 22.1, 'STOP'] ---
Emergency shutdown at batch 3.
```

4 Logic Trace

1. **State Setup:** Initialize `shutdown_triggered = False`.
2. **Outer Loop:** Iterates through `telemetry_stream` by batch index using `range(len())`.
3. **Inner Loop:** Scans individual data points inside the active batch:
   - i. **Noise (`"ERR"`):** Prints an error note and calls `continue` to bypass further checks.
   - ii. **Interrupt (`"STOP"`):** Emergency shutdown, flips `shutdown_triggered` to `True`, and executes an immediate `break`.
   - iii. **Delta Check:** Evaluates

$$|current - previous|$$

for adjacent numeric readings in the same batch, flagging a spike if the difference exceeds 5.0.

4. **Flag Check:** Following the inner loop, the outer loop evaluates `shutdown_triggered`. If `True`, a second `break` terminates the main processing pipeline.

5. **Normal Completion:** The outer loop's `else` block executes only if all batches process without encountering a `"STOP"` signal.

5 Rolling Delta Check

The Rolling Delta check compares consecutive numeric readings within the same batch.
For Batch 2:

$$|35.5 - 30.2| = 5.3$$

Since

$$5.3 > 5.0,$$

the program detects a spike between 30.2 and 35.5.
For the next readings:

$$|40.1 - 35.5| = 4.6$$

Since

$$4.6 < 5.0,$$

this transition is not classified as a spike.
Therefore, Batch 2 produces exactly one spike detection.

6 Program Output

The following output was obtained during execution of the `anomaly_auditor.py` program:

```
--- Auditing Batch 0: [22.5, 23.0, 22.8]---
Audit Complete: No system-wide failures
--- Auditing Batch 1: [25.1, 'ERR', 24.9]---
Noise ignored at Batch 1 (ERR)
Audit Complete: No system-wide failures
--- Auditing Batch 2: [30.2, 35.5, 40.1]---
Anomaly Detected at Batch 2 : 35.5
spike detected at batch id2:30.2 -> 35.5 Delta5.3
```

```
Anomaly Detected at Batch 2 : 40.1
Audit Complete: No system-wide failures
--- Auditing Batch 3: [22.0, 22.1, 'STOP']---
Emergency Shutdown at Batch 3.
```

7 Conclusion

The `anomaly_auditor` efficiently processes telemetry streams via nested control flow, correctly ignoring noise (`"ERR"`), flagging threshold anomalies, and detecting rolling spikes. Upon encountering a `"STOP"` signal, the system flips the `shutdown_triggered` flag to propagate an immediate `break` across both loops. This prevents further batch processing and bypasses the outer loop's `else` clause to guarantee a clean emergency termination.